BAMS

Business Management & Accounting System

System Architecture • Functional Specification • Database Blueprint • Implementation Roadmap

Technology baseline
Core PHP 8.4+ (no framework) • MySQL 8 • PDO • Bootstrap 5 • JavaScript

Prepared as an implementation-ready design specification

Purpose

BAMS is designed for a company to manage sales, product purchasing, inventory, employees, payroll, expenses, customers, suppliers, cash and bank accounts, and financial reporting in one connected application. Business events should post consistent stock and accounting records so owners can understand revenue, cost, profit, cash and liabilities without duplicate data entry.

Scope note: This document defines the target architecture and implementation plan. It is not a claim that the software has already been built, tested, or certified for statutory accounting compliance.

Document map

- 1. Executive summary and objectives
- 2. Architecture and technology stack
- 3. Roles and access control
- 4. Functional modules
- 5. Business workflows
- 6. Accounting model and calculation rules
- 7. Database blueprint
- 8. Core data relationships
- 9. Core PHP project structure
- 10. Security and data integrity
- 11. Reports and dashboard
- 12. Testing and acceptance criteria
- 13. Deployment and operations
- 14. Implementation roadmap
- 15. Decisions and assumptions

At a glance

Area	Design decision
Application model	Modular monolith: one integrated Core PHP application
Database	MySQL 8 / InnoDB, relational schema and foreign keys
Financial model	Double-entry accounting ledger; posted entries must balance
Inventory	Perpetual inventory with weighted-average costing as the initial default
Security	Server-side permissions, organization scoping, CSRF protection, audit trail
Deployment	HTTPS; web root restricted to public/; private storage and backups

1. Executive summary and objectives

BAMS is an integrated business management and accounting application for small and medium-sized businesses. It connects operational modules to a common accounting ledger and inventory movement model. The initial implementation should prioritize correctness, traceability, and maintainability over unnecessary complexity.

Business objectives

- Record sales, purchases, receipts, supplier payments, expenses and payroll in one place.
- Track inventory quantities, stock movements, cost of goods sold and inventory value.
- Calculate sales revenue, gross profit, operating profit, net profit and cash balances using defined rules.
- Track customer receivables and supplier payables, including partial settlements.
- Provide role-specific access for owners, accountants, sales, purchasing, inventory and HR staff.
- Preserve the history of posted financial records through controlled reversals and adjustments.

Non-goals for the first release

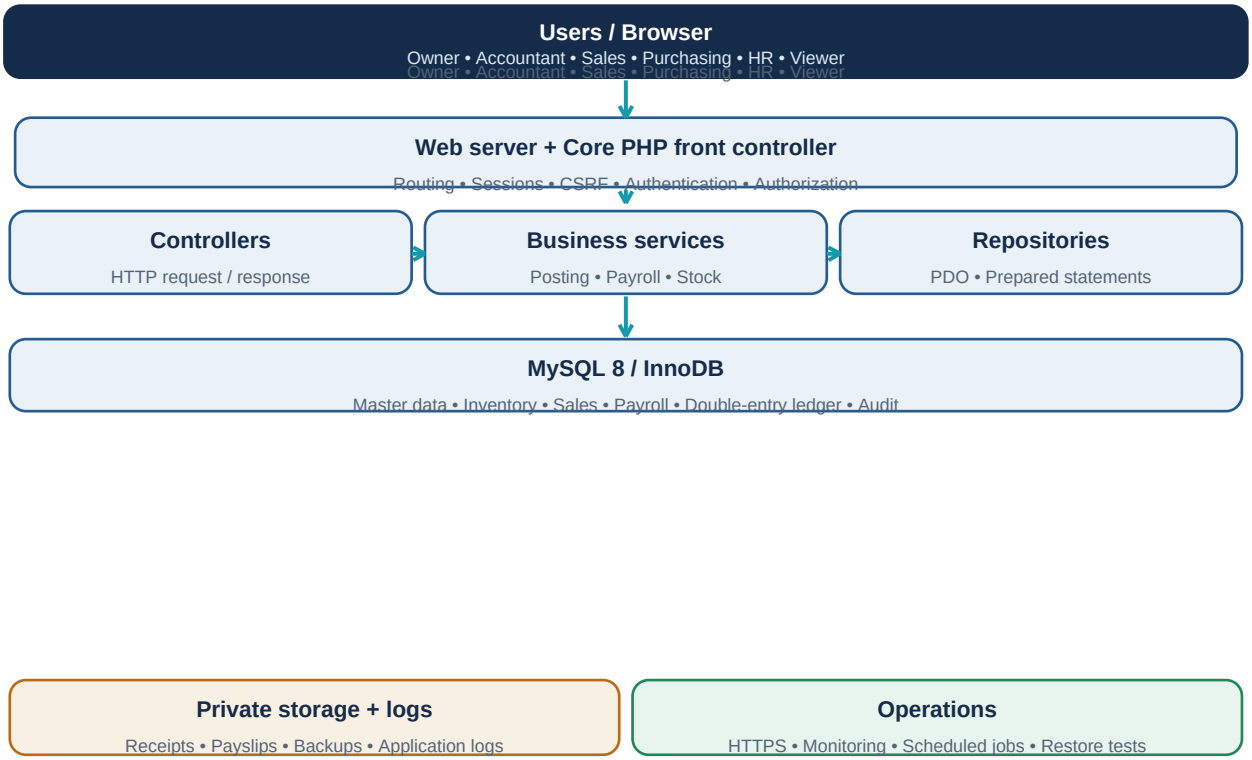
- Microservices, event streaming, or multiple databases without a demonstrated need.
- Automatic compliance with every country's tax, payroll or statutory reporting rules.
- Bank integrations, payment gateways, biometric attendance or e-commerce integrations unless explicitly added to scope.

Product principle

Operational events are the source of business activity; posted accounting entries are the source of financial reporting. A sale, payment, stock movement or payroll run must not be represented only as a manually edited dashboard number.

2. Architecture and technology stack

Recommended architecture: a modular monolith. Modules share one application and database, but each module has clear controllers, services, repositories, validation and permissions. This keeps deployment straightforward while supporting future expansion.



Technology baseline

Layer	Technology / approach
Backend	Core PHP 8.4+; no Laravel or other PHP framework
Frontend	HTML5, CSS3, Bootstrap 5, vanilla JavaScript
Database	MySQL 8 with InnoDB and utf8mb4
DB access	PDO, prepared statements, explicit transactions
Charts	Chart.js or equivalent client-side chart library
Authentication	PHP sessions, password_hash/password_verify, secure cookies
Web server	Apache or Nginx with PHP-FPM; HTTPS required
Testing	Unit/integration tests and browser end-to-end tests

Request lifecycle

1. Browser request reaches the public front controller and router.
2. Authentication middleware verifies the session and account state.
3. Company-context middleware resolves an organization the user is authorized to access.
4. Permission middleware checks the specific action and resource.
5. Controller validates input and calls a business service.
6. Service applies business rules and uses repositories through PDO.

7 Database transaction commits or rolls back as a unit.

8 Application returns the response and writes appropriate audit events.

3. Roles and access control

Use role-based access control (RBAC), with permissions enforced on the server for every request. Hiding a menu item is not authorization. Sensitive permissions should be separate, so creating an invoice does not automatically grant approval, refund, payroll access or financial-period closing.

Role	Typical access
Owner / Director	Company-wide overview, financial reports, approvals, configuration
Administrator	User and operational setup; sensitive permissions granted explicitly
Accountant	Ledger, reconciliation, invoices, expenses and financial statements
Sales Manager	Sales, customers, permitted refunds and sales reports
Sales Employee	Create permitted sales and record allowed customer receipts
Purchase Manager	Purchase orders, suppliers, bills and goods receipts
Inventory Manager	Products, warehouses, stock movements and stock reports
HR Manager	Employee records and approved HR workflows
Payroll Officer	Prepare payroll; approval can be assigned to another authorized user
Auditor / Viewer	Read-only access to specifically authorized records

Permission key examples

Module	Example permissions
Sales	sales.view, sales.create, sales.post, sales.refund
Purchases	purchases.view, purchases.create, purchases.receive, purchases.post
Inventory	inventory.view, inventory.adjust, inventory.transfer
Payroll	payroll.view, payroll.prepare, payroll.approve, payroll.pay
Expenses	expenses.create, expenses.approve, expenses.pay
Accounting	accounting.journal.create, accounting.journal.post, accounting.period.close
Reports	reports.sales.view, reports.inventory.view, reports.financial.view
Administration	users.manage, roles.manage, settings.manage, audit.view

Separation of duties

For material risks, separate preparation and approval. Examples include payroll posting, large refunds, manual journal entries, stock adjustments and period closing. The organization should configure approval thresholds and keep the approver identity and timestamps.

4. Functional modules

4.1 Dashboard

- KPIs: net sales, recognized revenue, cost of goods sold, gross profit, operating expenses, payroll expense, net profit and cash/bank balances.
- Operational alerts: low stock, overdue customer invoices, unpaid supplier bills, pending approvals and salary payments.
- Charts: sales trend, purchases, expense breakdown, profit trend, top-selling products and inventory value.
- Filters: today, week, month, year and custom date range; metrics must follow company timezone and posting status.

4.2 Sales management

- Customers, quotations/orders where required, sales invoices, invoice items, discounts, applicable taxes, due dates and payment terms.
- Cash, bank, mobile financial service and credit sales; full or partial receipts and allocation to invoices.
- Sales returns, refunds, credit notes, invoice printing and customer statements.
- Posting validates prices, stock, customer, permissions and accounting period; stock, receivable/cash and accounting entries are committed atomically.

4.3 Purchasing and suppliers

- Purchase orders, goods receipts, supplier bills, supplier payments, payment allocation and purchase returns.
- Goods receipt and supplier invoice may happen on different dates; support partial receipts and partial bills if needed.
- Track supplier balances and due dates; distinguish inventory purchases from immediate operating expenses.

4.4 Products and inventory

- Products, SKUs/barcodes, categories, units, warehouses, reorder levels and status.
- Stock receipts, sales issues, returns, transfers, adjustments and movement history.
- Initial costing recommendation: perpetual weighted-average cost; configure policy before production and apply it consistently.
- Prevent negative stock by default; use row locks and atomic updates to protect concurrent sales.

4.5 Employees and payroll

- Employee records, departments, salary structures, allowances, bonuses, deductions and authorized advances.
- Payroll period, draft calculation, review, approval, posting, payslips and payment tracking.
- Restrict salary data to explicitly authorized HR/payroll roles; keep sensitive fields out of general reports.

4.6 Expenses

- Expense categories, vendors, dates, amounts, payment accounts, references, receipt attachments and approvals.
- Separate recorded, approved, posted and paid states.
- Support rent, utilities, transport, marketing, office supplies, maintenance, bank fees and other configurable categories.

4.7 Cash and bank

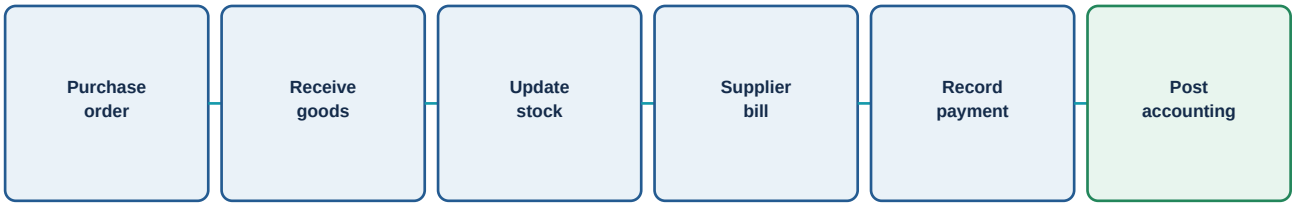
- Multiple cash/bank accounts, receipts, withdrawals, transfers, statements and reconciliation.
- Transfers affect the source and destination accounts but do not create revenue or expense.

4.8 Accounting

- Chart of accounts, journal entries and lines, general ledger, trial balance, receivables/payables, inventory accounting and period controls.
- Profit and loss, balance sheet and cash flow statements generated from posted entries.
- Every posted journal must balance; correction uses reversal/adjustment entries rather than silent deletion.

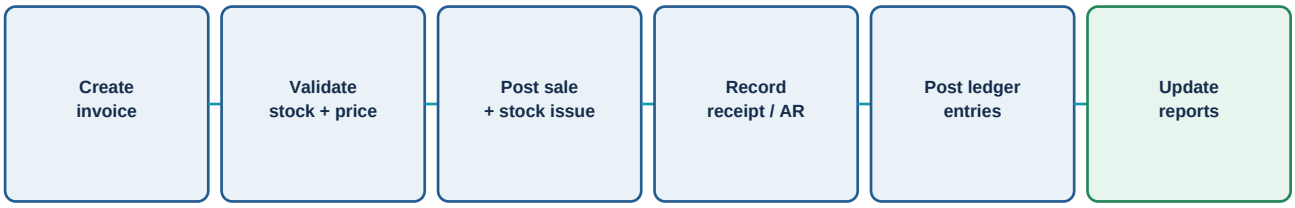
5. Business workflows

5.1 Purchase workflow



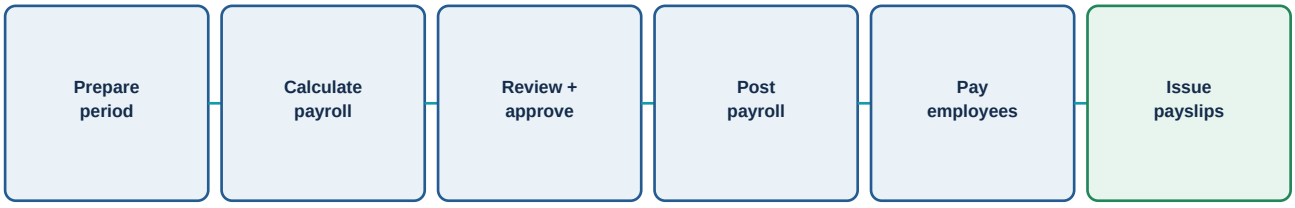
Purchase orders are commitments and do not automatically increase inventory. Goods receipts increase stock when goods are accepted. Supplier bills establish or update payables and inventory cost or expense according to the item and accounting policy. Payments settle the payable and reduce cash/bank.

5.2 Sales workflow



A posted sale records recognized revenue according to the configured policy, the applicable receivable or payment, and cost of goods sold/inventory reduction. If any required posting fails, roll back the entire operation.

5.3 Payroll workflow



Payroll posting normally recognizes salary expense and a salary payable. Actual payment later reduces the liability and cash/bank. If the business pays immediately, both steps may occur together but remain traceable.

5.4 Month-end close

1. Review customer and supplier aging and unallocated receipts/payments.
2. Reconcile cash, bank accounts, stock quantities and inventory valuation.
3. Review payroll, expenses, depreciation, accruals and manual journal entries as applicable.
4. Generate trial balance and investigate any imbalance or unusual movement.
5. Produce profit and loss, balance sheet and cash flow reports.
6. Close the period with authorized approval and prevent unauthorized backdated postings.

6. Accounting model and calculation rules

BAMS should use a double-entry accounting foundation even when the first release offers a simple user experience. Business modules generate the appropriate journal entries through a central accounting service; users should not manually enter the same financial event in multiple modules.

6.1 Core calculations

Metric	Definition
Net sales	Gross sales – discounts – sales returns, according to policy
Recognized revenue	Revenue earned for the period, excluding taxes collected for authorities where applicable
Cost of goods sold (COGS)	Inventory cost assigned to the goods sold
Gross profit	Net recognized sales revenue – COGS
Operating profit	Gross profit + other operating income – operating expenses
Net profit	Recognized income – recognized expenses, including applicable non-operating costs and taxes
Accounts receivable	Posted customer invoices – applied receipts and credits
Accounts payable	Posted supplier bills – applied payments and credits
Inventory value	Remaining eligible quantities × assigned inventory cost

6.2 Example sale

Assume products are sold for BDT 10,000 and their inventory cost is BDT 6,000, with no tax or discount for this illustration.

Account	Debit	Credit
Cash or accounts receivable	BDT 10,000	—
Sales revenue	—	BDT 10,000
Cost of goods sold	BDT 6,000	—
Inventory	—	BDT 6,000
Totals	BDT 16,000	BDT 16,000

Gross profit for this sale is BDT 4,000. If the customer has not paid, the debit is accounts receivable rather than cash. This illustrates why revenue, cash receipt and profit are distinct measures.

6.3 Accounting rules

- Every posted journal entry must satisfy total debits = total credits.
- A purchase of inventory is generally an asset acquisition; COGS is recognized when inventory is sold, subject to the chosen accounting policy.
- Customer receipts settle receivables and are not automatically new revenue.
- Supplier payments settle payables and are not automatically new expenses.
- Loan proceeds, owner capital and owner withdrawals must not be classified as sales revenue or operating expense by default.
- Taxes, depreciation, accruals, advances and statutory payroll treatment require explicit policies for the target jurisdiction.

7. Database blueprint

The schema should use InnoDB foreign keys, DECIMAL for monetary values, explicit status fields, indexes for common reports and unique business identifiers. Each module owns its operational records; the accounting ledger receives balanced postings from those modules.

Domain	Recommended tables
Company & security	organizations, users, organization_members, roles, permissions, user_roles, role_permissions, user_sessions
Sales	customers, sales_invoices, sales_invoice_items, sales_returns, sales_return_items, customer_receipts, receipt_allocations
Purchases	suppliers, purchase_orders, purchase_order_items, goods_receipts, goods_receipt_items, supplier_bills, supplier_bill_items, supplier_payments, payment_allocations
Inventory	products, product_categories, units, warehouses, stock_movements, stock_adjustments, stock_transfers, inventory_cost_layers
Employees & payroll	departments, employees, salary_structures, payroll_runs, payroll_items, salary_payments, employee_advances
Expenses & banking	expense_categories, expenses, expense_approvals, financial_accounts, account_transfers, bank_reconciliations
Accounting & control	chart_of_accounts, journal_entries, journal_lines, accounting_periods, audit_logs, attachments, company_settings

7.1 Required conventions

- Use BIGINT UNSIGNED primary keys or another consistent ID strategy; include organization_id on company-owned records.
- Use composite foreign keys where appropriate to prevent cross-company relationships.
- Use DECIMAL(19,4) for money/quantities where precision is required; define currency precision and rounding rules.
- Store invoice numbers unique per company and generate them safely under concurrency.
- Use created_at, updated_at, created_by, posted_by and posted_at where appropriate.
- Use restrictive deletes for financial records; use status transitions and reversals instead.
- Keep uploaded files in private storage; store metadata and access controls in the database.

7.2 Accounting tables

Table	Purpose / key fields
chart_of_accounts	organization_id, account_code, name, account_type, parent_id, status
journal_entries	organization_id, entry_number, entry_date, source_type, source_id, status, created_by, posted_by
journal_lines	journal_entry_id, account_id, debit, credit, description, party references
accounting_periods	organization_id, start_date, end_date, status, closed_by, closed_at
audit_logs	organization_id, actor_user_id, action, entity_type, entity_id, metadata, created_at

The final production schema must add exact types, indexes, check constraints, foreign keys, migration order and seed data. The table list in this document is a database blueprint, not an executable SQL migration.

8. Core data relationships

The following relationship map shows the main conceptual links. Detailed cardinalities and constraints should be encoded in migrations and verified with integration tests.

Parent / source	Relationship	Child / destination
organizations	1 to many	users / organization_members
organizations	1 to many	customers, suppliers, products, employees
customers	1 to many	sales_invoices and customer_receipts
sales_invoices	1 to many	sales_invoice_items
products	1 to many	sales_invoice_items and stock_movements
suppliers	1 to many	supplier_bills and supplier_payments
supplier_bills	1 to many	supplier_bill_items
employees	1 to many	payroll_items and salary_payments
expenses	many to one	expense_categories and financial_accounts
business source document	0/1 to many	journal_entries and journal_lines
journal_entries	1 to many	journal_lines
financial_accounts	1 to many	receipts, payments and transfers
users	many to many	roles through user_roles

Posting consistency

- Use one service to post a source document and its ledger entries inside one database transaction.
- Store the source document type and ID on accounting entries for traceability, plus a unique source-posting key to prevent duplicates.
- Use idempotency keys for retryable payment/posting requests.
- For concurrent stock operations, lock the relevant stock balance rows or use an atomic conditional update.
- Do not trust organization_id submitted by the browser; resolve it from the authenticated user's authorized company context.

9. Core PHP project structure

Use a small custom front controller and router rather than putting database queries and business logic directly in page files. The structure below is a recommendation; it is not a framework requirement.

```
bams/ ■■■ app/ ■ ■■■ Controllers/ ■ ■■■ Middleware/ ■ ■■■ Services/ ■ ■■■ Repositories/
■ ■■■ Validators/ ■ ■■■ Policies/ ■ ■■■ Support/ ■■■ config/ ■■■ database/ ■ ■■■
migrations/ ■ ■■■ seeds/ ■■■ resources/ ■ ■■■ views/ ■ ■ ■■■ layouts/ ■ ■ ■■■
dashboard/ ■ ■ ■■■ sales/ ■ ■ ■■■ purchases/ ■ ■ ■■■ inventory/ ■ ■ ■■■ payroll/ ■ ■
■■■ expenses/ ■ ■ ■■■ accounting/ ■ ■ ■■■ reports/ ■ ■■■ css/ ■ ■■■ js/ ■■■
routes/web.php ■■■ storage/ ■ ■■■ logs/ ■ ■■■ private-uploads/ ■■■ tests/ ■■■ bin/ ■■■
public/ ■ ■■■ index.php ■ ■■■ assets/ ■■■ bootstrap.php ■■■ .env.example ■■■ .gitignore
■■■ README.md
```

Layer responsibilities

Layer	Responsibility
Controllers	HTTP input/output; no complex financial logic
Middleware	Authentication, company context, CSRF and permission checks
Services	Business rules, posting orchestration and transaction boundaries
Repositories	PDO queries and persistence; prepared statements only
Validators	Server-side field and business input validation
Policies	Resource-level authorization and sensitive operation rules
Views	Presentation only; escape dynamic output
Migrations / seeds	Versioned schema changes and controlled reference data

Configuration secrets must come from environment or protected configuration outside public access. Never commit production credentials to source control.

10. Security and data integrity

Risk area	Required controls
Authentication	password_hash/password_verify, generic login errors, session regeneration, secure cookies, idle/absolute timeouts
Authorization	Server-side RBAC and resource-level policies on every request
CSRF / XSS	CSRF tokens for state changes; context-aware output escaping
SQL injection	PDO prepared statements; no concatenated user input in SQL
Tenant/company data	Resolve company context from membership; composite constraints and scoped queries
Duplicate posting	Unique source-posting key, idempotency token, database transaction
Concurrent stock	Row locks or conditional updates; atomic stock movement posting
Financial integrity	Balanced journals, immutable posted records, reversal/adjustment workflow
Uploads	Private storage, allowlisted MIME/extensions, size limits, random names, authorization
Operations	HTTPS, least privilege, backups, monitoring, patching and restore drills

Audit event minimums

- Authentication success/failure and sensitive account changes.
- Create, approve, post, cancel, reverse or refund financial documents.
- Payroll access and changes to salary structure or employee payment.
- Role and permission changes, exports, period closing and configuration changes.
- Record actor, organization, action, entity, timestamp, request/correlation ID and minimal metadata; avoid logging passwords, tokens or unnecessary personal data.

Financial immutability

Draft documents may be edited under policy. Posted documents should be corrected through explicit reversals, credit notes or adjustment entries. Database permissions and application services should prevent routine hard deletes of posted financial data.

11. Reports and dashboard definitions

Report	Primary data and notes
Sales summary	Posted sales invoices less returns/credits; filter by date, customer and product
Purchase summary	Supplier bills and accepted goods receipts; distinguish order, receipt and bill dates
Inventory on hand	Stock movements by product and warehouse; include as-of date
Inventory valuation	Quantity and assigned cost using the selected costing method
Customer aging	Open receivables by due date and applied receipts/credits
Supplier aging	Open payables by due date and applied payments/credits
Payroll summary	Posted payroll expense, liabilities and payment status; restricted access
Expense summary	Posted expenses by category, vendor, account and period
Profit and loss	Recognized income and expenses from posted journal lines
Balance sheet	Assets, liabilities and equity as of the report date
Cash flow	Cash/bank movements classified under the configured policy
Trial balance	Debit and credit balances of ledger accounts; totals must agree

Dashboard KPI rules

- Show selected date range and currency on every financial dashboard.
- Separate sales, recognized revenue, cash received, gross profit, net profit and bank/cash balance.
- Exclude draft/cancelled documents from posted financial KPIs.
- Apply returns, credits, taxes and accounting-period policies consistently.
- Show a clear empty state where there is no data; do not fabricate sample totals in production.
- For large datasets, aggregate in SQL with indexes and pagination; never load every transaction into the browser.

12. Testing and acceptance criteria

Test scenario	Acceptance result
Cash sale posting	Invoice, stock issue, cash receipt and balanced journals commit together
Credit sale	Receivable is created; cash does not increase until receipt is posted
Purchase receipt	Accepted quantity increases stock; cost and source document are traceable
Supplier payment	Payable and cash/bank decrease by the allocated payment
Payroll posting	Approved payroll expense and liabilities reconcile to employee items
Sales return	Stock and accounting effects follow return condition and policy
Reversal	Original record remains; compensating entries are balanced and linked
Duplicate submit/retry	No duplicate invoice, receipt or posting is created
Concurrent sale	Stock cannot be oversold under the configured stock policy
Permission check	Unauthorized role cannot access page, endpoint or protected record
Company isolation	User cannot read or alter another organization's records
Financial reports	Trial balance balances and reports reconcile to posted journal lines
Backup restoration	Database and required private files can be restored and verified

Test layers

- Unit tests: calculations, tax/discount rounding, costing, permission policies and allocation logic.
- Integration tests: PDO repositories, migrations, posting services and rollback behavior.
- End-to-end tests: login, create sale, receive purchase, payroll approval and financial reports.
- Concurrency tests: duplicate requests, stock locks, invoice numbering and simultaneous posting.
- User acceptance testing: sample scenarios agreed with the business owner/accountant.

Launch gate

Do not launch until opening balances, stock quantities, receivables/payables and sample financial statements reconcile; critical security tests pass; backups are restorable; and the business owner approves the accounting policy.

13. Deployment and operations

1. Provision a supported Linux server or managed hosting environment with a maintained PHP version and MySQL 8-compatible database.
2. Configure HTTPS, domain, web server and PHP-FPM; set the document root to the project's public/ directory.
3. Create a least-privilege database account; use protected environment variables for credentials.
4. Run versioned migrations and controlled seed scripts; create the initial owner using a one-time setup command.
5. Configure private uploads, log rotation, error monitoring, scheduled jobs and database backups.
6. Deploy to staging and test representative sales, purchase, payroll and accounting scenarios.
7. Import opening balances and stock through validated import tools; reconcile against source records.
8. Obtain approval, deploy production, monitor the first reporting period and verify backups.

Backup and recovery

- Automate encrypted database backups and separately back up private uploaded documents.
- Define recovery point objective (RPO) and recovery time objective (RTO) with the business owner.
- Perform periodic restore drills; a backup is not proven until a restore has been tested.
- Keep production logs free of secrets and restrict access to backups and payroll documents.

Operational monitoring

- Track application errors, failed jobs, database availability, disk usage, response time and suspicious login activity.
- Alert on failed backup jobs, unexpected posting failures and repeated accounting reconciliation discrepancies.
- Apply security updates and maintain a documented deployment/rollback process.

14. Implementation roadmap

A small experienced team may plan approximately 12–16 weeks for a carefully scoped initial release. This is a planning estimate, not a guarantee; local tax requirements, attendance, multiple warehouses, integrations and data migration can increase scope.

Phase	Target	Deliverables / exit criteria
1. Foundation	Weeks 1–2	Requirements, schema migrations, project structure, routing, PDO, logging, configuration and CI baseline
2. Access + master data	Weeks 3–4	Login, RBAC, company settings, customers, suppliers, products, accounts, opening balances
3. Sales + purchasing	Weeks 5–7	Invoices, receipts, supplier bills, goods receipts, returns, stock ledger and costing
4. Payroll + expenses	Weeks 8–9	Employees, salary structures, payroll runs, approvals, expense posting and payment tracking
5. Accounting + reports	Weeks 10–11	Chart of accounts, journals, ledger, P&L;, balance sheet, cash flow, reconciliation
6. Stabilize + launch	Weeks 12–16	Integration/E2E tests, security review, data reconciliation, backups, user acceptance and deployment

Recommended delivery order

1. Build database migrations, authentication, permissions and company settings.
2. Build chart of accounts and a tested accounting posting service.
3. Build products, stock movements, suppliers and purchasing.
4. Build sales invoices, receipts, returns and customer balances.
5. Build expenses, employees, payroll and salary payments.
6. Build reports and dashboard from posted records.
7. Complete security hardening, automated tests, restore drills and launch readiness review.

15. Decisions and assumptions

This specification uses the following defaults to keep the design coherent. Confirm or adjust them before the production schema and detailed UI are finalized.

Decision	Default assumption	Why it matters
Backend	Core PHP only; no Laravel or other PHP framework	Matches the project constraint
Frontend	Bootstrap 5 + HTML/CSS/JavaScript	Responsive and practical for admin dashboards
Database	MySQL 8 / InnoDB	Transactions, foreign keys and relational reporting
Accounting	Double-entry ledger foundation	Makes financial reports auditable and reconcilable
Inventory costing	Perpetual weighted-average	General-purpose starting point; confirm with accountant
Currency	Company-configurable; BDT is the example	Currency precision and tax handling affect calculations
Business type	General trading/service business	Industry-specific features may require extensions
Warehouses	Schema supports multiple locations if needed	Can simplify UI for a single warehouse at first
Tax/payroll rules	Configured for the jurisdiction and reviewed before launch	Statutory requirements cannot be assumed
Deployment	Single application / modular monolith	Lower initial operational complexity

Key implementation warnings

- This document is a target design, not a completed codebase, executable schema or audited accounting product.
- Exact SQL migrations must specify every field, foreign key, unique constraint, index, check and deletion rule.
- Financial posting, inventory costing, tax treatment and payroll calculations require tests and business/accounting review.
- Do not treat example figures, sample records or dashboard mockups as actual company financial data.
- Use a staged rollout with reconciled opening balances before moving real business activity into production.

Suggested next deliverables

1. Complete MySQL 8 schema and versioned migration files.
2. Core PHP project skeleton with router, PDO connection, sessions, CSRF and RBAC middleware.
3. Accounting posting service and ledger tests.
4. Sales, purchase, stock, payroll and expense module implementation specifications.
5. Responsive dashboard and module UI pages.
6. Test plan, deployment guide and data migration templates.

End of specification • BAMS Business Management & Accounting System